

Ransomware Detection on IoT Edge Using Optimized Deep Representation Learning

Yaser Almasri¹, MD Sakibul Islam¹, Mohammed Elnour¹, M Waleed Shinwari¹, Farid Binbeshr¹, Ashraf Mahmoud¹, and Muhammad Imam¹

¹Affiliation not available

February 13, 2025

Ransomware Detection on IoT Edge Using Optimized Deep Representation Learning

Yaser Almasri, MD Sakibul Islam, Mohammed Elnour, M. Waleed Shinwari, Farid Binbeshr, Ashraf Mahmoud, Muhammad Imam

Abstract—The increasing adoption of edge computing in Internet of Things (IoT) and Cyber-Physical Systems (CPS) environments has made these systems attractive targets for ransomware attacks, as sensitive data is often processed at the edge. As IoT nodes are typically resource-constrained, traditional ransomware detection solutions are unsuitable for these environments. In this paper, we propose a novel lightweight ransomware detection framework for IoT edge devices using an optimized Deep Representation Learning Model (DeepLSE). Our model leverages advanced compression techniques, including quantization and pruning, to reduce the model size and power consumption, making it suitable for deployment on resource-limited edge nodes. We utilize the CICAndMal 2017 dataset to train and evaluate our model, achieving an impressive 99.05% accuracy. Moreover, our model demonstrates significant advantages over conventional CNN and DNN approaches, with an inference time of only 6 milliseconds and a power consumption of 0.149 watts—demonstrating superior efficiency compared to other platforms, which consume significantly more power. The framework also includes a cloud-based device management system that handles edge node alerts and enables real-time control of infected nodes. Our research provides an efficient, real-time solution for ransomware detection at the IoT edge, ensuring both high accuracy and low resource usage.

Index Terms—Ransomware Detection, IoT network, Auto-encoder, Latent-space encoding, Deep Learning, Representation Learning, Resource-aware model.

I. INTRODUCTION

THE Internet of Things (IoT) has revolutionized technology, connecting devices across domains such as smart homes, healthcare, autonomous vehicles, and industrial systems. This rapid expansion brings benefits like efficiency and innovation but also introduces significant security challenges, as IoT networks handling critical data and operations are prime targets for cyber-attacks [1], [2]. The power, memory, and computation limitations of many IoT devices prevent the deployment of robust security measures and limit the security layers to generic implementations that are easy attack targets [3]. Ransomware has emerged as a major threat, exemplified

by incidents like WannaCry, which rapidly exploited vulnerable networks. Proactive measures, including Ransomware Detection Systems that analyze network traffic, are crucial. AI-assisted systems enable early detection at the network edge, preventing the spread of ransomware [4], [5].

Edge computing enhances IoT network performance by bringing computational resources closer to data sources, reducing latency, and enabling real-time processing [6]. However, the proximity of resources demands adaptive security measures. Machine learning (ML) and deep learning (DL) techniques show promise in detecting anomalies [7]–[10], but their deployment in resource-constrained IoT environments is challenging. References [3], [7], [9] highlight energy consumption, runtime, and challenges of ML/DL models, including large model sizes exceeding IoT storage limits and high energy demands impacting battery-powered devices. Therefore, lightweight approaches are necessary to balance detection accuracy with low resource consumption. This makes lightweight ransomware defense critical for IoT devices in both industrial and consumer applications. Developing effective detection methods that work in real-world scenarios is a pressing need.

However, several critical research gaps persist. First, many existing solutions are evaluated using high-resource systems, neglecting the practical limitations of IoT devices [11]. IoT-centric research often overlooks the unique challenges posed by resource-constrained devices, such as microcontrollers, which are common in IoT networks. Additionally, while cloud-centric solutions for ransomware detection are prevalent, real-time edge-based detection is still underexplored, possibly due to the relative infancy of widespread critical edge computation as a desired alternative. Moreover, there is a lack of practical testing and benchmarking on various datasets, which hinders the generalization and effectiveness of existing solutions [3], [8], [12]. Addressing these gaps, this research develops an optimized ransomware detection system for IoT edge devices, balancing detection accuracy and resource efficiency. Techniques like Quantization, Pruning, and Clustering offer potential for compressing deep learning models, though the extent of compression achievable without compromising accuracy remains a challenge. So the following research questions motivate this study:

- 1) **RQ1:** Are deep learning models feasible for deployment on edge devices considering their large size?
- 2) **RQ2:** How do deep learning models perform when deployed on edge devices?
- 3) **RQ3:** What is the inference time for deep learning models on edge devices?

Yaser Almasri, MD Sakibul Islam and Mohammed Elnour are with the Department of Computer Engineering at King Fahd University of Petroleum and Minerals, Dhahran, 31261, Saudi Arabia (email: {g201930090, g202321310, g202318430}@kfupm.edu.sa).

M. Waleed Shinwari is with the Department of Computer Engineering and Interdisciplinary Research Center for Industrial Manufacturing and Robotics (IRC-IMR) at King Fahd University of Petroleum and Minerals, Dhahran, 31261, Saudi Arabia (email: mohammad.shinwari@kfupm.edu.sa).

Farid Binbeshr, Ashraf Mahmoud and Muhammad Imam are with the Department of Computer Engineering and Interdisciplinary Research Center for Intelligent Secure Systems (IRC-ISS) at King Fahd University of Petroleum and Minerals, Dhahran, 31261, Saudi Arabia (email: {farid.binbeshr, ashraf, mimam}@kfupm.edu.sa).

Corresponding author: mimam@kfupm.edu.sa

- 4) **RQ4:** How much power do deep learning models consume during operation on edge devices?

In this research, we focus on tiny machine-learning deployment, to optimize model performance on devices like the ESP32. This approach is crucial for edge nodes in IoT networks, where optimized solutions are necessary to minimize resource consumption—such as model size, inference time, and power usage—while still achieving high detection accuracy. By addressing these challenges, we aim to ensure real-time ransomware detection in IoT networks, improving resilience against evolving cyber threats.

Our research makes several key contributions:

- 1) We propose an Optimized Deep Representation Learning (DeepLSE) model for ransomware detection on IoT edge nodes that effectively balances high detection accuracy with low resource consumption.
- 2) We extensively explored different machine and deep learning models, studying their performance alongside resource consumption to identify their drawbacks and effectiveness for IoT environments.
- 3) We introduce a hybrid architecture for the hierarchical management of edge nodes, facilitating scalable and efficient operations within IoT networks.
- 4) Our solution is evaluated on practical IoT edge platforms, such as the ESP32, showcasing its applicability and performance in terms of detection rate, resource usage, and overall efficiency.

Overall, this research addresses the critical need for robust ransomware defenses while considering the resource constraints of IoT edge environments, contributing to the development of feasible and scalable security solutions.

The remainder of this paper is structured as follows: Section II reviews related work. Section III outlines the proposed architecture, methodology, model implementations and optimization, and experimental setup, including the dataset, pre-processing steps, and feature selection methods. Section IV presents the results, while Section V discusses the findings. Finally, Section VI concludes the paper and outlines future directions.

II. LITERATURE REVIEW

Ransomware detection has been extensively studied, with methods broadly categorized into non-AI-based techniques and AI-based approaches such as machine learning and deep learning. Non-AI-based techniques rely on predefined rules, signatures, or behavioral patterns to identify ransomware activities. In contrast, AI-based methods can model under-represented or highly nonlinear dependencies that traditional methods often miss or grossly approximate. By leveraging data-driven models, they effectively overcome the limitations of static and rule-based techniques for ransomware detection.

A. Non AI-based Ransomware Detection

Decoy-based techniques, widely used for ransomware detection, rely on honeypots or honey files to lure ransomware and enable early detection. Moore [13] proposed a honeypot

system for Windows networks with decoy files to monitor activities and alert administrators. Hernández et al. [14] introduced a system deploying honey files to detect and block ransomware while initiating recovery. Gómez-Hernández et al. [15] advanced this with R-Locker, which scattered trap files and dynamically blacklisted malicious entities. Despite effectiveness, these methods can still be subverted by clever ransomware behavior and still generate a lot of alarms at scale [16].

Behavioral methods detect ransomware by analyzing file modifications, system behavior, and anomalies. UNVEIL, proposed by Kharaz et al. [17], created artificial environments to identify tampering and detect zero-day ransomware. Continella et al. [18] introduced ShieldFS, which flagged abnormal file system activities and reversed ransomware effects. Although effective, dynamic environments often cause false positives. Signature-based methods remain useful for known threats but are ineffective against evolving zero-day ransomware.

Junaid et al. [19] developed StateDroid, a modeling app that attacks state transitions. Alzahrani et al. [20] proposed RanDroid, detecting ransomware through app textual strings. Chen et al. [21] introduced RansomProber, trained on a comprehensive ransomware dataset, achieving high accuracy. These models are highly specific in their targets (UI elements, state transitions, or specific data) and might not be directly applicable across the diverse IoT ecosystems.

B. Machine Learning & Deep Learning-based Ransomware Detection

Several studies have leveraged machine learning (ML) and deep learning (DL) techniques for ransomware detection, focusing on diverse models and datasets. Ensemble methods combining Decision Tree (DT), Random Forest (RF), Gradient Boosting (GB), and Support Vector Machine (SVM) and AdaBoost achieved 91.75% accuracy on the SCIREL-20M dataset, highlighting balanced performance [9], though it did not address real-time deployment. Singh et al. [22] proposed a hybrid model using DT, SVM, kNN, ANN, and SGD, achieving a near-perfect accuracy of 99.98%, but the absence of real-time functionality remained a limitation. Similarly, Almashhadani et al. [23] achieved 99.7% accuracy using session-based features but faced challenges adapting to varying traffic patterns. A layered random network (LRN) by Prachi et al. [8], integrating RF, SVM, and kNN, achieved 99.99% accuracy, although its static implementation posed challenges for edge-based deployment.

Berrueta et al. [24] integrated RF, kNN, and Bayesian models, achieving 97.92% accuracy while optimizing computational overhead via feature reduction. However, the need for real-time validation remains for practical deployment. Maimo et al. [25] combined OC-SVM and NB to dynamically detect and mitigate ransomware, achieving 99.99% accuracy with real-time applicability, making it a strong candidate for edge IoT environments. From a network traffic analysis perspective, Albin Ahmed et al. [26] explored ML models on mobile ransomware datasets, emphasizing the necessity of lightweight and real-world deployable models for edge devices. Similarly,

a CNN utilizing the VGG-16 architecture achieved 96.80% accuracy on VirusShare's dataset [7], although its constrained real-time utility limits use in latency-sensitive contexts.

C. Lightweight Ransomware Detection

The HARD-Lite model by Woralert et al. [27] enables real-time ransomware detection on resource-constrained IoT devices using Performance Monitoring Units (PMUs) for hardware event data and Long Short-Term Memory (LSTM) models for anomaly detection. Its key features include the K-LEB tool for low-overhead data collection and encrypted communication, ensuring minimal performance impact. HARD-Lite exhibits high accuracy, rapid detection, and low false positives, but faces challenges with scalability, hardware variability, and evasion tactics. The ACOLBSR method from Sreelaja et al. [28] improves traditional signature-based detection inefficiencies by employing Ant Colony Optimization (ACO) combined with a binary search for computational efficiency, achieving 100% accuracy in controlled environments. However, it lacks mechanisms for evolving ransomware variants and has limited applicability in dynamic IoT ecosystems.

Zhang et al.'s Ranker [29] introduces a dual-layered detection mechanism that analyzes kernel-level behavior for early ransomware detection, achieving 99.43% accuracy with minimal resource use. However, it mainly targets Windows systems and needs adaptation for IoT and improved scalability. Wang et al.'s CanCal framework [16] employs a lightweight multi-layered approach to minimize false positives and resource overheads, integrating decoy file monitors and semantic ransom note detectors. Evaluated on over 3.32 million endpoints, CanCal demonstrates strong scalability and performance but faces challenges with evasion tactics and limited response measures.

Mofidi et al.'s L-IDS framework [30] combines Trusted Execution Environments (TEE) with lightweight machine learning for IoT ransomware detection, optimizing resource use while isolating threats, though facing scalability and compatibility challenges. Ahmed et al. [30] introduced a Weighted minimum Redundancy maximum Relevance (WmRmR) technique for early crypto-ransomware detection in Industrial IoT, achieving high accuracy but lacking extensive validation. Tariq et al. [31] developed a hybrid method using Gradient Boosting Machines and Lasso Regression to detect ransomware anomalies in IIoT networks, focusing on detection. Celdran et al. [32] proposed a policy-based framework for resource-efficient IoT ransomware detection, with an enhanced version [33] improving scalability. Liu et al.'s ICEAP [34] detects ransomware through network traffic analysis but lacks IoT optimizations. Lastly, Novak et al. [35] examine Large Language Models for ransomware detection from logs and traffic, but their computational demands through the use of LLMs pose scalability issues needing real-world validation.

III. METHODOLOGY

A. Proposed System Architecture

The proposed system architecture is designed to detect ransomware in IoT edge environments. It includes several steps:

Dataset acquisition, Data preprocessing, Model training and evaluation, optimization for deployment, Real-time detection on IoT edge nodes, and centralized monitoring and alerting via a cloud server. Below we discuss each component briefly:

- 1) **Dataset Acquisition and Preprocessing** The process begins with acquiring the CICAndMal dataset, which includes benign and ransomware-infected network traffic. Preprocessing involves feature selection, binarization of labels, addressing class imbalance, and converting categorical features to numerical formats to ensure clean and relevant data for training.
- 2) **Model Architecture (DeepLSE)** The core of the system is the Deep Representation Learning Model. After preprocessing, the model is trained and evaluated using a testing set to classify network traffic as benign or ransomware, focusing on accuracy and reliability in detection.
- 3) **Optimized Model for Deployment** To deploy on resource-constrained IoT devices like ESP32, optimization methods reduce the model's size and computational demands, resulting in a lightweight, efficient model suitable for edge nodes.
- 4) **IoT Layer for Real-Time Detection** The optimized model is deployed on IoT edge nodes to analyze incoming network traffic and classify it. Multiple edge nodes operate concurrently for comprehensive monitoring. Detection logs are uploaded to a centralized cloud server for further analysis.
- 5) **Cloud Server for Centralized Storage and Alerts** A cloud server acts as the central hub for storing logs and providing hierarchical monitoring. If ransomware is detected on the edge device, the server triggers alerts to the admin/user interface, ensuring scalability and management of edge nodes.
- 6) **Admin/User Control Interface** This interface allows users to monitor the system, view logs, receive alerts, and control edge nodes. If ransomware is detected on a node, administrators can take immediate actions like isolating infected nodes to prevent damage.

By integrating these components, the proposed system architecture offers a robust, scalable, and resource-efficient solution for ransomware detection in IoT environments. The combination of an optimized deep learning model, real-time edge detection, and centralized monitoring ensures effective mitigation of ransomware threats in modern IoT networks.

B. Ransomware Dataset Evaluation & Selection

Ransomware detection models depend on datasets that capture the unique behaviors of ransomware attacks. Common sources include public repositories like VirusTotal and Malshare, as well as proprietary datasets from organizations like Microsoft. When selecting a dataset, it's crucial to ensure it specifically targets ransomware and is suitable for IoT environments. Datasets can be categorized into three types: static features datasets analyze files without execution (easy to create but limited in behavioral insights), dynamic features datasets monitor file behavior during execution (providing

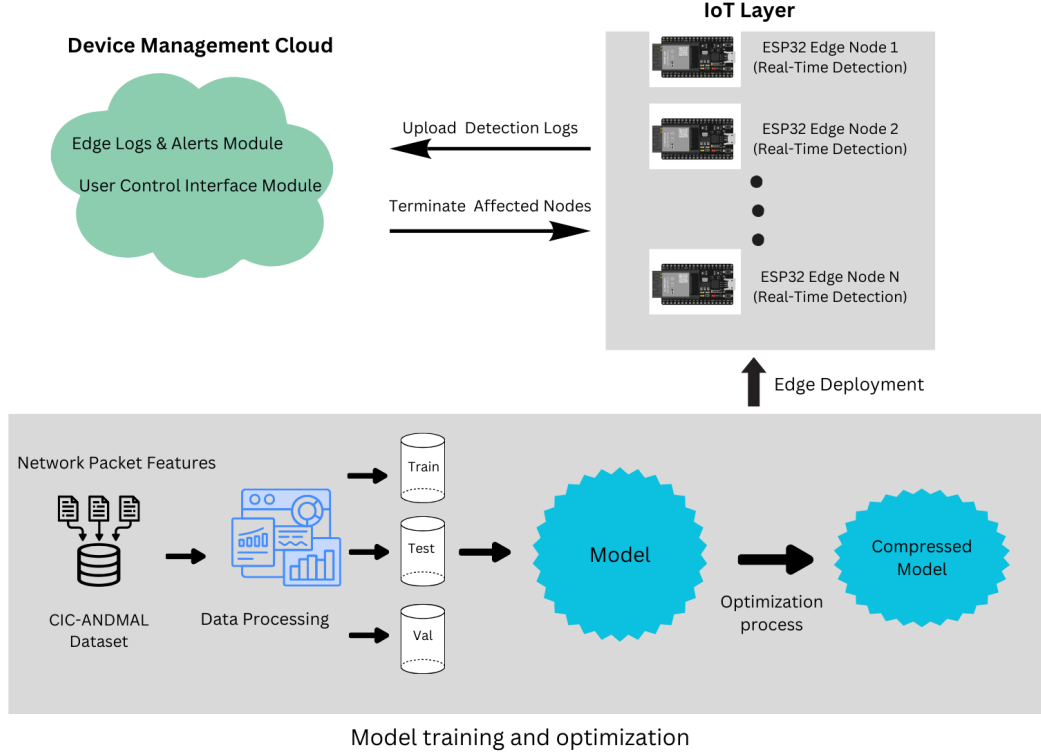


Fig. 1. Proposed system architecture. The dataset is first preprocessed; then, the DeepLSE model is trained and optimized (compressed) for deployment on Edge IoT nodes. Alarms are sent to the cloud system, for hierarchical management, to monitor and isolate affected nodes.

detailed insights but requiring more resources), and network features datasets examine network traffic (ideal for real-time detection despite complexity).

For our research, we selected the CICAndMal dataset from the Canadian Institute for Cybersecurity, focusing on network traffic features [36]. This dataset is well-suited for edge environments, encompassing a wide range of samples from various ransomware families and detailed network traffic data, with 84 standard network-flow features essential for detecting ransomware behavior. Among the dataset versions, the 2017 version is most appropriate, utilizing real network traffic compared to the emulated environments of the 2019 and 2020 versions.

C. Data Processing

The data processing phase comprised stages such as feature examination, label binarization, handling class imbalance, feature conversion, and preparation for model training. The dataset includes real-world network traffic data from smartphones infected with ten ransomware families and benign traffic, featuring 84 network-flow attributes. During processing, twelve irrelevant attributes with constant zero values were removed, including flags and averages related to network behavior, resulting in a total of 72 features remaining for analysis. A total of 4,500 random samples were selected from each of the 10 ransomware classes, resulting in 45,000 ransomware samples. All samples from these classes were labeled as (1), while 43,091 benign samples were labeled as (0).

Categorical features like FlowID, source IP, and destination IP were converted to numerical values for effective model processing. After preprocessing, the dataset was divided into training (80%) and validation/testing sets (20%).

D. Model Architecture

In this paper, we propose a deep representation learning method following this paper [37]. To our knowledge, no other paper has applied representation learning for detecting threats like ransomware or malware. It utilizes an autoencoder-inspired neural network to transform high-dimensional network traffic features into a compressed latent space, where a classifier is applied to differentiate between benign and ransomware samples.

The DeepLSE model comprises an encoder module (E) that compresses the high-dimensional feature vector (X) into a lower-dimensional representation known as the latent space (LV). This encoder reduces redundancy and extracts meaningful patterns that are critical for classification. The latent space serves as the model's core, containing the essential features compressed by the encoder. The decoder module (D) reconstructs the original feature vector ($\hat{X}$) from the latent space representation. Meanwhile, the classifier module (C) operates directly on this latent space to distinguish between benign and ransomware inputs. By being constrained by both reconstruction fidelity (through the decoder) and classification accuracy (via the classifier), the model ensures that the encoder effectively captures the essential information of the

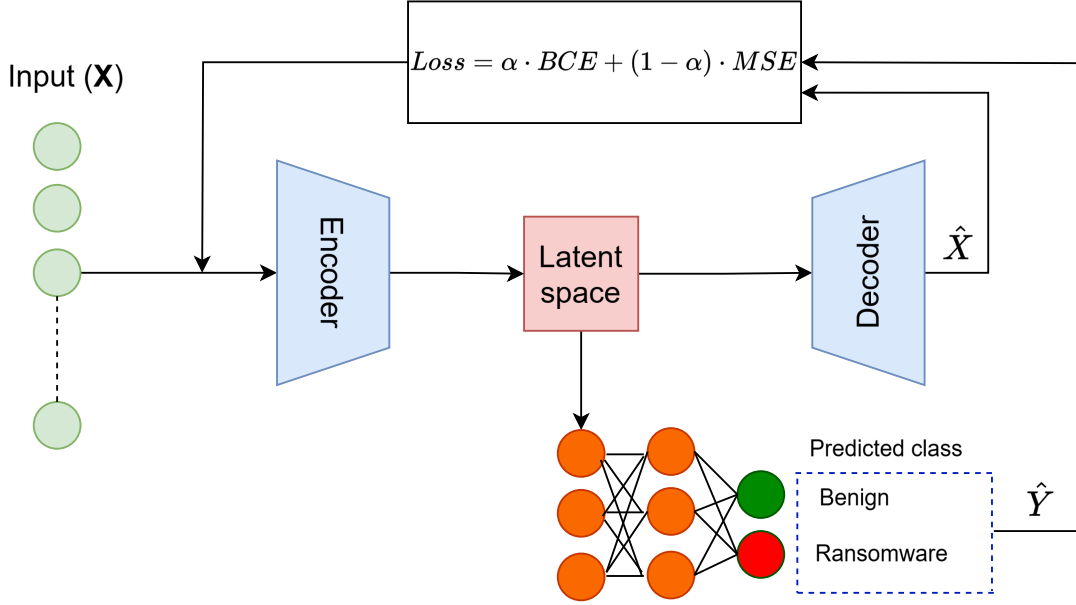


Fig. 2. DeepLSE Model Architecture: The encoder compresses input features into a latent space, where a classifier differentiates between benign and ransomware classes. The decoder ensures effective reconstruction of the input, guided by the loss function.

input features. Designed to balance these objectives, the latent space remains both compact and highly discriminative for classification tasks.

A combination of Binary Cross Entropy (BCE) and Mean Squared Error (MSE) losses, adjusted via α , guide the model to balance accuracy and generalization. The DeepLSE model optimizes two objectives simultaneously using a combined loss function defined as follows:

$$L_{\text{DeepLSE}} = \alpha \cdot L_{\text{Class}} + (1 - \alpha) \cdot L_{\text{Recon}} \quad (1)$$

where

$$L_{\text{Class}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (2)$$

and

$$L_{\text{Recon}} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2 \quad (3)$$

where y_i represents the binary indicator (0 or 1) for the true class label, and p_i denotes the predicted probability of the observation. x and x_i represent the input and reconstructed feature vectors, respectively, each of size N . BCE is utilized by the classifier to minimize the error in predicting binary class labels (benign vs. ransomware), ensuring that the latent space representation is both discriminative and effective for classification. In contrast, MSE is employed by the auto-encoder to minimize the reconstruction error between the input (X) and the reconstructed output ($\hat{X}$). This approach ensures that the latent space captures sufficient and essential information for accurate reconstruction, which subsequently leads to better classification results.

The loss mixing parameter (α) balances between classification accuracy and reconstruction fidelity. A higher value of α (e.g., 0.99) prioritizes classification accuracy, optimizing the latent space for maximum separation between benign

and ransomware classes. However, this can result in slightly higher reconstruction errors, and therefore low classification accuracy. On the other hand, a lower α (e.g., 0.01) emphasizes reconstruction accuracy, ensuring the latent space retains more comprehensive information but with potentially reduced classification performance.

Figure 2 summarizes the proposed model architecture. The advantages of using an autoencoder-based neural network include its ability to automatically compress features into the most important ones, eliminating the need for feature filtration techniques like PCA or Chi-square. Moreover, the decoder penalizes the latent space, compelling the encoder to focus on the most significant features, thus optimizing classification accuracy. To prevent overfitting, batch normalization, and dropout are applied at the encoder and decoder layers.

E. Model Optimization for Edge Deployment

The optimization techniques for the DeepLSE model aim to reduce size and enhance efficiency for resource-constrained edge devices, utilizing methods from the TensorFlow Lite library:

- 1) **Quantization** This technique reduces numerical precision, typically from 32-bit floating-point (FP32) to 16-bit (FP16) or 8-bit integers (INT8), and is implemented post-training.
- 2) **Pruning** Pruning eliminates parameters with minimal impact on predictions, allowing for more effective compression while retaining original size and latency, optimizing storage without compromising performance.
- 3) **Weight Sharing (Clustering)** This method reduces unique weight values by clustering similar weights and sharing a centroid for each cluster:
 - Grouping weights into N clusters.

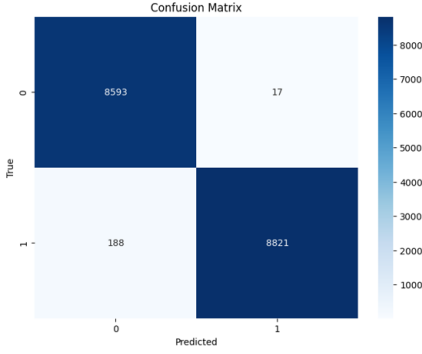


Fig. 3. DeepLSE Confusion Matrix.

- Sharing centroid values among weights in each cluster.

These steps compress the model, balancing performance and size, making it ideal for devices with limited resources. We utilized TensorFlow Lite's default compression option, optimizing for accuracy, inference time, and model size, along with quantization to reduce size and improve inference time at the cost of some accuracy.

F. Experimental Setup

The model is implemented on Google Colab using TensorFlow, optimized with TensorFlow Lite, and deployed on ESP32 devices. An MB102 board supplies 3.3 volts to the ESP32, while a multimeter measures the current consumed by the device. The IoT edge device features a 240 MHz CPU, 448 KB ROM, and 520 KB SRAM. For cloud integration, we used the Cumulocity IoT cloud with MQTT protocol for communication, enabling remote monitoring and management of inference results. Inference time (T_{invoke}) was measured as the time taken by the TensorFlow Lite interpreter to process input data and generate output, with multiple runs on the ESP32 to assess latency and precision. After training and optimization, the compressed and quantized model was deployed, testing its feasibility in a resource-constrained environment while ensuring efficient performance. Accuracy, precision, recall, and F1-score metrics were used for model evaluation, and their equations can be found in [37].

IV. RESULT ANALYSIS

A. Performance of DeepLSE Model

The DeepLSE model demonstrated strong performance across multiple evaluation metrics, effectively detecting ransomware traffic in real-time, as shown in Table I. It achieved a peak accuracy of 99.05%, with a precision of 98.00% and a recall of 100.00%, highlighting its effectiveness in minimizing false positives while perfectly identifying ransomware.

The F1-score of 99.00% reflects a robust balance between precision and recall. The confusion matrix in Figure 3 shows only 188 false negatives and 17 false positives from 19,000 test samples, validating the model's capability to accurately identify ransomware, critical for edge detection where false negatives can spread. The ROC curve analysis in Figure 4

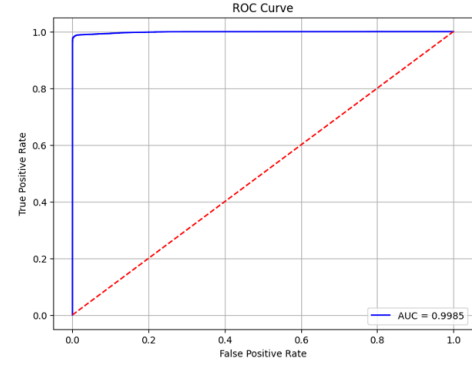


Fig. 4. DeepLSE ROC curve.

yielded an AUC of 0.9985, indicating exceptional discrimination between benign and ransomware traffic. Figure 5 illustrates the effective convergence of training and validation loss.

Latent space visualization in Figure 6 shows distinct non-overlapping clusters for benign and ransomware samples, evidencing the model's ability to differentiate classes. The BCE encourages separation in the latent space, while the MSE preserves essential data information, leading to a compact and informative representation.

The Accuracy vs. Alpha analysis in Figure 7, with latent vector (LV) sizes from 3 to 7, and alpha values from 0 to 1 show consistent performance trends. Accuracy surpassed 96% as alpha approached 0.10, peaking at 0.75, where classification and reconstruction losses were optimized. This underscores the importance of tuning alpha for an optimal trade-off between classification accuracy and reconstruction efficiency, highlighting the model's robustness to hyperparameter changes.

B. Performance Comparison with Existing Literature

The performance of the DeepLSE model was compared with several existing approaches to validate its effectiveness. Table I shows that DeepLSE outperforms Ayub et al.'s [9] ensemble of decision tree (DT), random forest (RF), gradient boosting (GB), and support vector machine (SVM), which achieved 91.75% accuracy, and Ciaramella et al.'s [7] CNN model with 96.80% accuracy.

Table II further compares DeepLSE with other state-of-the-art models tested on datasets with network-related features. DeepLSE excels in ransomware detection, achieving 99.05% accuracy, 98.00% precision, 100.00% recall, and 99.00% F1-score. Unlike studies by Shaukat et al. [38] and Yang et al. [39], which used decision trees and random forests, DeepLSE leverages deep learning for superior results. Although Mercaldo et al.'s [40] model achieved slightly higher accuracy, it often relied on specialized datasets, making it less comparable to general-purpose network scenarios. This comparison establishes DeepLSE as a robust solution for network-based feature detection in diverse IoT and CPS environments.

C. Resource Consumption of Different Models

Table III highlights the resource consumption of various models trained on network feature datasets. The DeepLSE

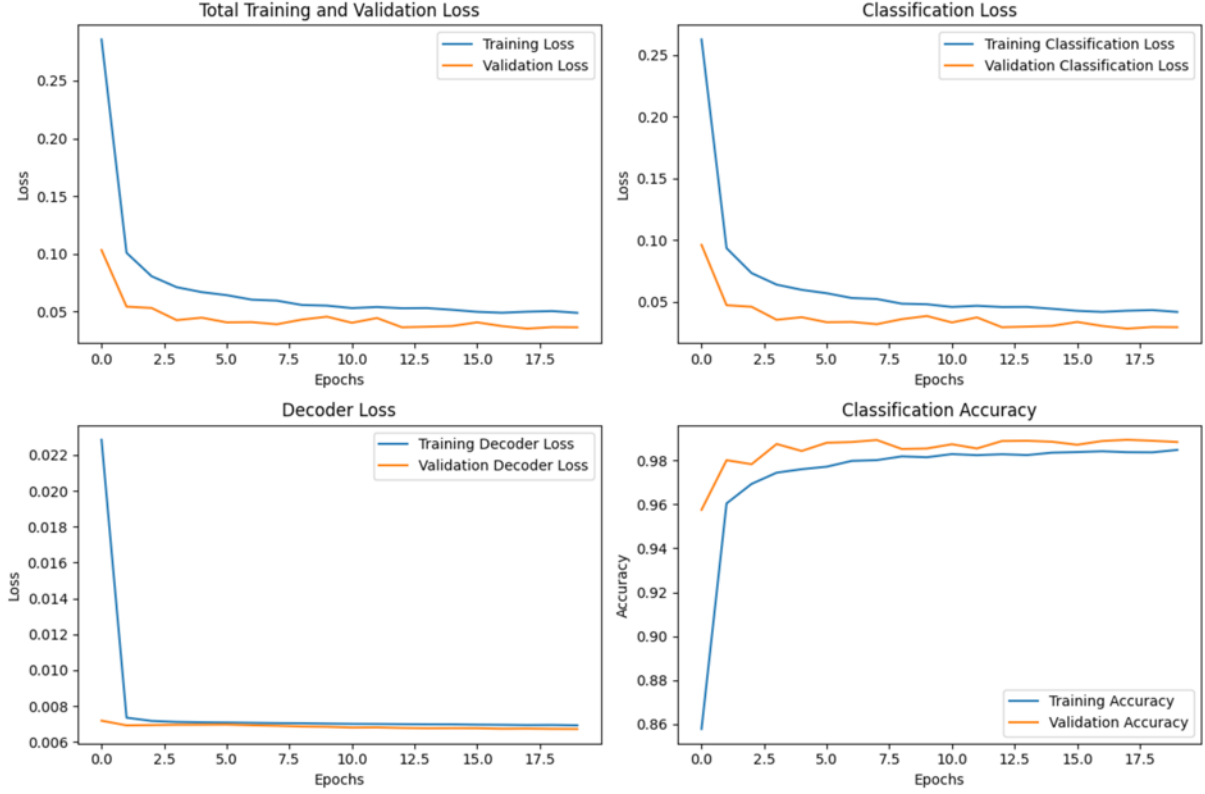


Fig. 5. Training and validation losses of the DeepLSE model. The decrease in classification loss as the decoder converges leads to higher accuracy, illustrating the model's effective functioning.

TABLE I
COMPARISON OF PROPOSED DEEPLSE MODEL WITH RELATED LITERATURE

Ref Study	Algorithms	Dataset	Real-time?	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
[9]	Ensemble: DT, RF, GB, SVM, Adaboost	SCIREL-20M (215 samples)	No	91.75	91.99	90.47	91.05
[7]	CNN, VGG-16	VirusShare (5000 samples)	No	96.80	97	96.90	-
[22]	DT, SVM, kNN, ANN, SGD	70 ransomware samples (31 families)	No	99.83	99.82	99.83	99.83
[8]	RF, SVM, KNN, LR, NB	50 ransomware samples (10 families)	No	99	99.20	98.90	98.80
[10]	DT, TensorFlow	70 ransomware samples (Hybrid Traffic Analysis)	Partial	99	99.70	100	99.87
[23]	DT, kNN, SVM, Discriminant Analysis	Session-based: 4128 Ransomware, 4128 benign time-based from MCFP project	Partial	99.88	99.76	100	99.88
[25]	OC-SVM, Naive Bayes	150,000 Samples from Clear Traffic	Yes	99.99	92.32	99.97	95.96
Proposed	DeepLSE	88,000 Samples from CIC-ANDMAL Dataset	Yes	99.05	98.00	100.00	99.00

model stands out with an initial size of 156.94 KB, which can be compressed to 25.67 KB and 16.52 KB through default compression and quantization options, respectively—about one-tenth of its original size. In contrast, models like the Optimized DNN and Optimized CNN remain large post-optimization, with quantized sizes of 213.05 KB and 301.13 KB, making them unsuitable for edge devices.

Table IV evaluates models suitable for edge devices like the ESP32. The DeepLSE model consumes only 45.4 mA (0.14982 watts) during inference, significantly lower than the

Optimized CNN, which consumes 60.5 mA (0.19965 watts). DeepLSE has an inference time of 6 ms, enabling near real-time ransomware detection and making it the fastest among the benchmarked models.

In contrast, the hybrid CNN-LSTM IDS [41], when evaluated on a Raspberry Pi, exhibited a peak power consumption of 6.12 W during prediction. Similarly, the lightweight IDS proposed by [42] consumed only 2.45 W but suffered from a higher detection latency of 6.07 seconds. These findings underscore the superior efficiency of DeepLSE in resource-

TABLE II
COMPARISON OF PROPOSED DEEPLSE MODEL WITH NETWORK FEATURE-BASED LITERATURES

Ref Study	Algorithms	Dataset Features	Feature Type	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
[38]	DT	IP, packets, ports, protocols, duration, etc.	Network	97.10	97.30	97.10	97.10
[39]	RF	Inflow burst lengths, interflow times, packet lengths, etc.	Network	87.00	86.00	87.00	87.00
[24]	RF, LibSVM, BayesNet, and Random Tree	TCP, HTTP, DNS, and NBNS traffic from PCAP files	Network	97.92	97.90	97.90	97.90
[40]	Neural Network	140 encrypted/non-encrypted files	Network	99.99	99.70	99.70	99.74
Proposed	DeepLSE	Flow ID, protocol, IP, packet stats, etc.	Network	99.05	98.00	100.00	99.00

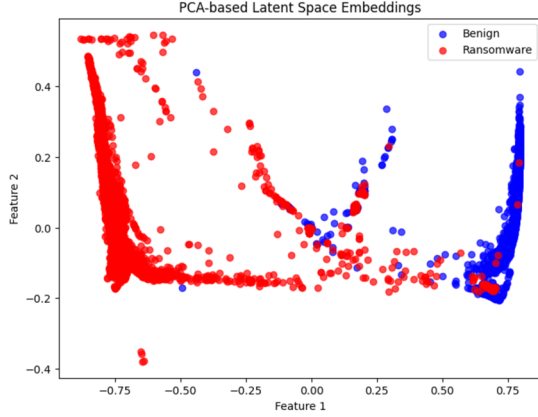


Fig. 6. Scatter plot of the PCA projection of the model's latent space.

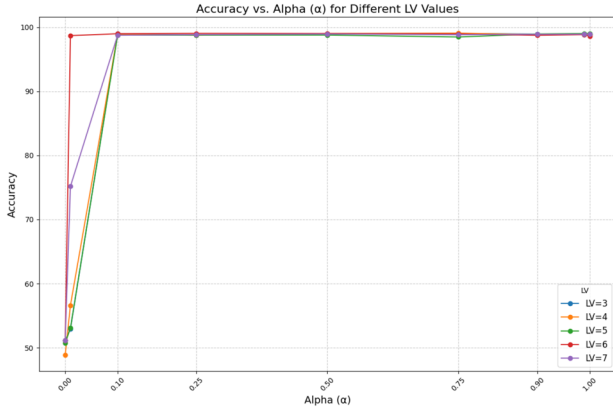


Fig. 7. Model tuning for different LV values. Model accuracy peaks when α value is equal to 0.75

constrained edge environments.

Figure 8 shows the detection log from the edge node sent to the cloud, providing a hierarchical view of benign and infected nodes for effective control. This architecture offers substantial benefits for Industrial Internet of Things (IIoT) systems, where effective monitoring is essential. Overall, the DeepLSE model excels in detection speed, low power consumption, and accuracy, making it ideal for resource-constrained IoT edge environments.

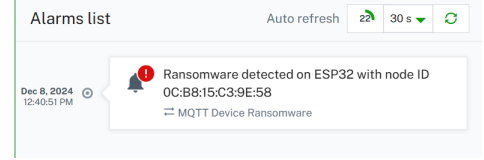


Fig. 8. Edge node detection logs in the cloud. Operations can be sent from the cloud to terminate an infected node based on its ID.

V. DISCUSSION

This study addresses ransomware detection on resource-limited IoT edge devices by proposing a lightweight model and optimized framework. The DeepLSE model demonstrates effective performance, achieving over 99% detection accuracy.

RQ1: Feasibility of Deep Learning on IoT Edge Devices

The research shows that DeepLSE can run on ESP32 devices, utilizing quantization and pruning to significantly reduce model size while maintaining high accuracy.

RQ2 & RQ3: Performance and Real-Time Detection

DeepLSE provides real-time detection with an inference time of 6 ms, meeting the requirements for ransomware detection on IoT devices.

RQ4: Low Power Consumption

The model consumes only 0.14982 W, making it suitable for battery-powered IoT devices and efficient in resource management.

Unlike previous studies that relied on high-resource platforms, our approach enables effective deep learning in edge environments, reducing memory and energy consumption while maintaining high precision, accuracy, and recall.

The proposed hierarchical architecture enhances the management of edge nodes, ensuring reliability and resilience, while the hybrid edge-cloud system improves scalability and adaptability for rapid response and resource management.

VI. CONCLUSION & FUTURE WORK

This research presents an optimized Deep Representation Learning (DeepLSE) model for ransomware detection on IoT edge devices. We successfully show that the model works well on devices like the ESP32, providing high detection accuracy while using minimal resources. By utilizing optimization techniques, we reduced the model size and maintained a high level of performance. The results highlight the potential of using deep learning in IoT environments with limited resources.

TABLE III
MODEL SIZE COMPARISON BETWEEN PROPOSED DEEPLSE AND EXISTING LITERATURE SOLUTIONS

Ref Study	Model	Initial Model Size (KB)	Compressed Model Size (KB)	Quantized Model Size (KB)	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Deployable on Edge Node ESP32
[43]	DNN	2535.32	829.29	224.57	99.09	98.00	100.00	99.00	No
[44]	1D-CNN	3516.79	1161.27	301.13	98.95	98.00	100.00	99.00	No
[43]	Optimized DNN	2398.26	788.95	213.05	99.08	98.00	100.00	99.00	Yes
[44]	Optimized CNN	1872.64	616.38	162.71	98.91	98.00	100.00	99.00	Yes
[45]	LSTM	467.22	122.89	54.45	99.07	98.00	100.00	99.00	Yes
Proposed	DeepLSE	156.94	25.67	16.52	99.05	98.00	100.00	99.00	Yes

TABLE IV
EDGE DEPLOYMENT PERFORMANCE COMPARISON OF PROPOSED DEEPLSE MODEL

Ref Study	Model	Current (mA)	Power (Watts)	Inference Time
[43]	Optimized DNN	50.2	0.16566	34 ms
[44]	Optimized CNN	60.5	0.19965	1014 ms
[45]	LSTM	47.8	0.15774	25 ms
Proposed	DeepLSE	45.4	0.14982	6 ms

Our approach also emphasizes real-time, edge-based solutions instead of relying on cloud systems, making it more efficient for IoT networks. However, this study has some limitations, particularly regarding the explainability of the model's predictions. Understanding how deep learning models arrive at their conclusions remains a significant challenge.

Future work consists of testing the DeepLSE model on various low-resource devices to assess its effectiveness across a broader range of practical IoT scenarios. Another area of interest is integrating federated learning to enhance privacy during model training. Finally, generative models, such as GANs, are considered to simulate zero-day attacks and better evaluate the model's resilience to emerging threats.

REFERENCES

- [1] M. Aboubakar, M. Kellil, and P. Roux, "A review of iot network management: Current status and perspectives," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 7, p. 4163–4176, Jul. 2022. [Online]. Available: <http://dx.doi.org/10.1016/j.jksuci.2021.03.006>
- [2] Y. Shah and S. Sengupta, "A survey on classification of cyber-attacks on iot and iiot devices," in *2020 11th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*. IEEE, 2020, pp. 0406–0413.
- [3] M. Islam, A. Yusuf, M. D. Gambo, and A. Y. Barnawi, "A novel few-shot ml approach for intrusion detection in iot," *Arabian Journal for Science and Engineering*, pp. 1–15, 2024.
- [4] D. W. Fernando, N. Komninos, and T. Chen, "A study on the evolution of ransomware detection using machine learning and deep learning techniques," *IoT*, vol. 1, no. 2, pp. 551–604, 2020.
- [5] T. Radhakrishna and N. E. Majd, "Edge computing ransomware detection in iot using machine learning," in *2024 International Conference on Computing, Networking and Communications (ICNC)*. IEEE, Feb. 2024, p. 244–248. [Online]. Available: <http://dx.doi.org/10.1109/ICNC59896.2024.10556351>
- [6] F. C. Andriulo, M. Fiore, M. Mongiello, E. Traversa, and V. Zizzo, "Edge computing and cloud computing for internet of things: A review," *Informatics*, vol. 11, no. 4, p. 71, Sep. 2024. [Online]. Available: <http://dx.doi.org/10.3390/informatics11040071>
- [7] G. Ciaramella, G. Iadarola, F. Martinelli, F. Mercaldo, and A. Santone, "Explainable ransomware detection with deep learning techniques," *Journal of Computer Virology and Hacking Techniques*, vol. 20, no. 2, p. 317–330, Sep. 2023. [Online]. Available: <http://dx.doi.org/10.1007/s11416-023-00501-1>
- [8] Prachi and S. Kumar, "An effective ransomware detection approach in a cloud environment using volatile memory features," *Journal of Computer Virology and Hacking Techniques*, vol. 18, no. 4, p. 407–424, Apr. 2022. [Online]. Available: <http://dx.doi.org/10.1007/s11416-022-00425-2>
- [9] M. A. Ayub, A. Siraj, B. Filar, and M. Gupta, "Rwarmor: a static-informed dynamic analysis approach for early detection of cryptographic windows ransomware," *International Journal of Information Security*, vol. 23, no. 1, p. 533–556, Sep. 2023. [Online]. Available: <http://dx.doi.org/10.1007/s10207-023-00758-z>
- [10] E. Berrueta, D. Morato, E. Magaña, and M. Izal, "Crypto-ransomware detection using machine learning models in file-sharing network scenarios with encrypted traffic," *Expert Systems with Applications*, vol. 209, p. 118299, 2022.
- [11] Z. A. Khan and P. Herrmann, "Recent advancements in intrusion detection systems for the internet of things," *Security and Communication Networks*, vol. 2019, p. 1–19, Jul. 2019. [Online]. Available: <http://dx.doi.org/10.1155/2019/4301409>
- [12] A. Huertas Celdrán, P. M. Sánchez Sánchez, J. von der Assen, D. Shushack, L. Perales Gómez, G. Bovet, G. Martínez Pérez, and B. Stiller, "Behavioral fingerprinting to detect ransomware in resource-constrained devices," *Computers and Security*, vol. 135, p. 103510, Dec. 2023. [Online]. Available: <http://dx.doi.org/10.1016/j.cose.2023.103510>
- [13] C. Moore, "Detecting ransomware with honeypot techniques," in *2016 Cybersecurity and Cyberforensics Conference (CCC)*, 2016, pp. 77–81.
- [14] J. Gómez-Hernández, L. Álvarez González, and P. García-Teodoro, "R-locker: Thwarting ransomware action through a honeyfile-based approach," *Computers and Security*, vol. 73, p. 389–398, Mar. 2018. [Online]. Available: <http://dx.doi.org/10.1016/j.cose.2017.11.019>
- [15] J. A. Gómez-Hernández, R. Sánchez-Fernández, and P. García-Teodoro, "Inhibiting crypto-ransomware on windows platforms through a honeyfile-based approach with r-locker," *IET Information Security*, vol. 16, no. 1, p. 64–74, Sep. 2021. [Online]. Available: <http://dx.doi.org/10.1049/ise2.12042>
- [16] S. Wang, F. Dong, H. Yang, J. Xu, and H. Wang, "Cancal: Towards real-time and lightweight ransomware detection and response in industrial environments," *arXiv preprint arXiv:2408.16515*, 2024.
- [17] A. Kharaz, S. Arshad, C. Mulliner, W. Robertson, and E. Kirda, "{UNVEIL}: A {Large-Scale}, automated approach to detecting ransomware," in *25th USENIX security symposium (USENIX Security 16)*, 2016, pp. 757–772.
- [18] A. Continella, A. Guagnelli, G. Zingaro, G. De Pasquale, A. Barengi, S. Zanero, and F. Maggi, "Shieldfs: a self-healing, ransomware-aware filesystem," in *Proceedings of the 32nd annual conference on computer security applications*, 2016, pp. 336–347.
- [19] M. Junaid, J. Ming, and D. Kung, "Statedroid: Stateful detection of stealthy attacks in android apps via horn-clause verification," in *Pro*

- ceedings of the 34th Annual Computer Security Applications Conference, 2018, pp. 198–209.
- [20] A. Alzahrani, A. Alshehri, H. Alshahrani, R. Alharthi, H. Fu, A. Liu, and Y. Zhu, “Randroid: Structural similarity approach for detecting ransomware applications in android platform,” in *2018 IEEE International Conference on Electro/Information Technology (EIT)*. IEEE, 2018, pp. 0892–0897.
 - [21] S. Chen, M. Xue, L. Fan, S. Hao, L. Xu, H. Zhu, and B. Li, “Automated poisoning attacks and defenses in malware detection systems: An adversarial machine learning approach,” *computers & security*, vol. 73, pp. 326–344, 2018.
 - [22] J. Singh, K. Sharma, M. Wazid, and A. K. Das, “Sinn-rd: Spline interpolation-envisioned neural network-based ransomware detection scheme,” *Computers and Electrical Engineering*, vol. 106, p. 108601, 2023.
 - [23] A. O. Almashhadani, D. Carlin, M. Kaiiali, and S. Sezer, “Mfmncs: A multi-feature and multi-classifier network-based system for ransomworm detection,” *Computers & Security*, vol. 121, p. 102860, 2022.
 - [24] E. Berrueta, D. Morato, E. Magaña, and M. Izal, “Open repository for the evaluation of ransomware detection tools,” *IEEE Access*, vol. 8, p. 65658–65669, 2020. [Online]. Available: <http://dx.doi.org/10.1109/ACCESS.2020.2984187>
 - [25] L. Fernandez Maimo, A. Huertas Celdran, A. L. Perales Gomez, F. J. Garcia Clemente, J. Weimer, and I. Lee, “Intelligent and dynamic ransomware spread detection and mitigation in integrated clinical environments,” *Sensors*, vol. 19, no. 5, p. 1114, 2019.
 - [26] A. Albin Ahmed, A. Shaahid, F. Alnasser, S. Alfaddagh, S. Binagag, and D. Alqahtani, “Android ransomware detection using supervised machine learning techniques based on traffic analysis,” *Sensors*, vol. 24, no. 1, p. 189, 2023.
 - [27] C. Woralert, C. Liu, and Z. Blasingame, “Hard-lite: A lightweight hardware anomaly realtime detection framework targeting ransomware,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2023.
 - [28] N. Sreelaja, “Ant colony optimization based light weight binary search for efficient signature matching to filter ransomware,” *Applied Soft Computing*, vol. 111, p. 107635, 2021.
 - [29] H. Zhang, L. Zhao, A. Yu, L. Cai, and D. Meng, “Ranker: Early ransomware detection through kernel-level behavioral analysis,” *IEEE Transactions on Information Forensics and Security*, 2024.
 - [30] F. Mofidi, S. Hounsino, and G. Bloom, “L-ids: A lightweight hardware-assisted ids for iot systems to detect ransomware attacks,” in *Proceedings of the 8th ACM/IEEE Conference on Internet of Things Design and Implementation*, 2023, pp. 464–465.
 - [31] U. Tariq, “Combatting ransomware in zephyros-activated industrial iot environments,” *Heliyon*, vol. 10, no. 9, 2024.
 - [32] A. H. Celdrán, P. M. Sánchez Sánchez, E. J. Scheid, T. Besken, G. Bovet, G. M. Pérez, and B. Stiller, “Policy-based and behavioral framework to detect ransomware affecting resource-constrained sensors,” in *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, 2022, pp. 1–7.
 - [33] A. H. Celdrán, P. M. S. Sánchez, J. von der Assen, D. Shushack, A. L. P. Gómez, G. Bovet, G. M. Pérez, and B. Stiller, “Behavioral fingerprinting to detect ransomware in resource-constrained devices,” *Computers & Security*, vol. 135, p. 103510, 2023.
 - [34] T.-M. Liu, D.-Y. Kao, and Y.-Y. Chen, “Loocipher ransomware detection using lightweight packet characteristics,” *Procedia Computer Science*, vol. 176, pp. 1677–1683, 2020.
 - [35] R. Novak, D. Rivera, and T. Haas, “Lightweight real-time ransomware mitigation in industrial networks using large language models.”
 - [36] A. H. Lashkari, A. F. A. Kadir, L. Taheri, and A. A. Ghorbani, “Toward developing a systematic approach to generate benchmark android malware datasets and classification,” in *2018 International Carnahan conference on security technology (ICCST)*. IEEE, 2018, pp. 1–7.
 - [37] S. Khan, “Deep-representation-learning-based classification strategy for anticancer peptides,” *Mathematics*, vol. 12, no. 9, 2024. [Online]. Available: <https://www.mdpi.com/2227-7390/12/9/1330>
 - [38] K. Shaikat, S. Luo, and V. Varadharajan, “A novel method for improving the robustness of deep learning-based malware detectors against adversarial attacks,” *Engineering Applications of Artificial Intelligence*, vol. 116, p. 105461, Nov. 2022. [Online]. Available: <http://dx.doi.org/10.1016/j.engappai.2022.105461>
 - [39] L. Yang, A. Ciptadi, I. Laziuk, A. Ahmadzadeh, and G. Wang, “Bodmas: An open dataset for learning based temporal analysis of pe malware,” in *2021 IEEE Security and Privacy Workshops (SPW)*. IEEE, May 2021. [Online]. Available: <http://dx.doi.org/10.1109/SPW53761.2021.00020>
 - [40] F. Mercaldo, “A framework for supporting ransomware detection and prevention based on hybrid analysis,” *Journal of Computer Virology and Hacking Techniques*, vol. 17, no. 3, p. 221–227, May 2021. [Online]. Available: <http://dx.doi.org/10.1007/s11416-021-00388-w>
 - [41] H. Alzahrani, T. Sheltami, A. Barnawi, M. Imam, and A. Yaser, “A lightweight intrusion detection system using convolutional neural network and long short-term memory in fog computing,” *Computers, Materials and Continua*, vol. 80, no. 3, p. 4703–4728, 2024. [Online]. Available: <http://dx.doi.org/10.32604/cmc.2024.054203>
 - [42] F. Aliyu, T. Sheltami, M. Deriche, and N. Nasser, “Human immune-based intrusion detection and prevention system for fog computing,” *Journal of Network and Systems Management*, vol. 30, no. 1, Oct. 2021. [Online]. Available: <http://dx.doi.org/10.1007/s10922-021-09616-6>
 - [43] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, “Deep learning approach for intelligent intrusion detection system,” *Ieee Access*, vol. 7, pp. 41 525–41 550, 2019.
 - [44] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, “End-to-end encrypted traffic classification with one-dimensional convolution neural networks,” in *2017 IEEE international conference on intelligence and security informatics (ISI)*. IEEE, 2017, pp. 43–48.
 - [45] S. Hochreiter, “Long short-term memory,” *Neural Computation MIT-Press*, 1997.